Assignment 3: Software-Defined Networking

COL7524: Advanced Networks
Deepak - 2025CSY7616
Pratik - 2025MCS2096

We used the two supplied topologies without changing their links, delays or host connections. The experiments below compare reactive and proactive rules, shortest-delay routing, and routing that reacts to link load. All results come from the saved Mininet runs included with this report.

1. Reactive and proactive flow management

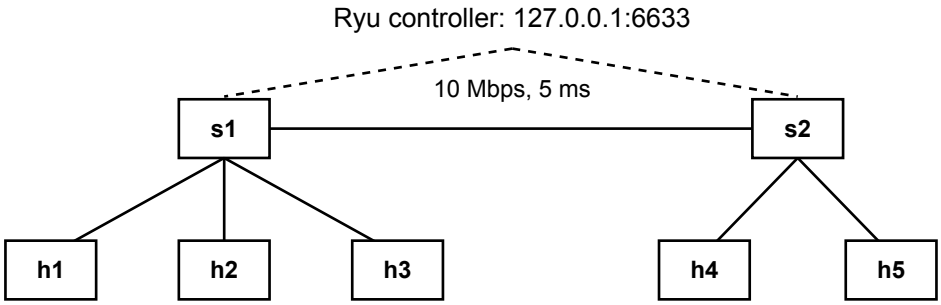


Figure 1. The two-switch topology used in Part 1. Dashed lines show controller connections.

The reactive controller learns the incoming port for each source MAC. An unknown destination is flooded; once its port is known, the controller installs a forwarding rule. The proactive controller already knows where all five hosts are connected, so it installs five destination rules and one broadcast rule on each switch before testing begins.

For each controller, we started a clean network, saved the initial tables, ran pingAll, and sent ten pings from h1 to h5. We then saved the final tables and counted the reactive controller's Packet-In log entries. The count covers the whole test sequence, including ARP and other non-LLDP Ethernet traffic handled by that application.

Measure	Reactive	Proactive
pingAll loss	0%	0%
Forwarding Packet-In events	101	0
Mean h1-h5 ping RTT (ms)	10.122	10.139
Initial rules per switch	1 table-miss rule	6 forwarding rules

The warmed-up RTTs are close because both switches then forward locally. The main difference is at setup: reactive forwarding needs controller decisions for initial traffic but installs rules only when needed. Proactive forwarding avoids that initial controller work, but needs advance knowledge and installs rules even for unused destinations. First-packet delay was not measured separately here.

The proactive count is supported by its flow tables: there is no output-to-controller rule. The before/after tables, Ryu output and successful ping summaries on pages 2-3 show this difference.

1.1 Reactive forwarding: recorded evidence

The excerpts on this page are read directly from results/p1-reactive.txt and results/p1-reactive-controller.log. Flow rows retain packet/byte counters, priority, match and action; only the cookie, duration and table number are omitted.

Commands used for controller and switch inspection

```
.venv/bin/ryu-manager --ofp-tcp-listen-port 6633 p1/reactive.py
sudo python3 p1_topo.py
ovs-ofctl -O OpenFlow13 dump-flows s1
ovs-ofctl -O OpenFlow13 dump-flows s2
```

The driver takes the first table dump before pingAll and the ten h1-h5 pings, then takes the final dump after those tests. It runs the same controller/topology and invokes these flow-table commands through Mininet.

Before tests: only the table-miss rule on each switch

```
s1: n_packets=28, n_bytes=2408, priority=0 actions=CONTROLLER:65535
s2: n_packets=28, n_bytes=2408, priority=0 actions=CONTROLLER:65535
```

Selected Ryu output

```
Switch 1 connected
Switch 2 connected
Packet-In: switch=1 src=00:00:00:00:00:01 dst=00:00:00:00:00:05 out=4
Packet-In: switch=2 src=00:00:00:00:00:01 dst=00:00:00:00:00:05 out=2
```

After tests: h1/h5 forwarding rules and their packet counters

```
s1: n_packets=13, n_bytes=1218, priority=1,in_port="s1-eth4",dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
s1: n_packets=9, n_bytes=882, priority=1,in_port="s1-eth1",dl_dst=00:00:00:00:00:05 actions=output:"s1-eth4"
s2: n_packets=10, n_bytes=980, priority=1,in_port="s2-eth2",dl_dst=00:00:00:00:00:01 actions=output:"s2-eth3"
s2: n_packets=13, n_bytes=1274, priority=1,in_port="s2-eth3",dl_dst=00:00:00:00:00:05 actions=output:"s2-eth2"
```

Connectivity and controller-workload results

```
PINGALL loss percent: 0.0
10 packets transmitted, 10 received, 0% packet loss, time 1804ms
rtt min/avg/max/mdev = 10.112/10.122/10.150/0.011 ms
Logged forwarding Packet-In events: 101
```

Initially, unmatched packets go to Ryu. The Packet-In lines show h1-to-h5 packets reaching the controller on both switches. The final tables then contain priority-1 unicast rules with nonzero counters: h1-to-h5 uses s1 port 4 and s2 port 2; replies use s2 port 3 and s1 port 1.

The initial table-miss counters are already nonzero because hosts generate background Ethernet traffic before the controlled ping tests. There are still no learned unicast rules in that first snapshot. The total Packet-In count includes that startup traffic, pingAll and the measured pings.

1.2 Proactive forwarding: recorded evidence

The excerpts on this page are read directly from results/p1-proactive.txt and results/p1-proactive-controller.log. Flow rows retain packet/byte counters, priority, match and action; only the cookie, duration and table number are omitted.

Commands used for controller and switch inspection

```
.venv/bin/ryu-manager --ofp-tcp-listen-port 6633 p1/proactive.py
sudo python3 p1_topo.py
ovs-ofctl -O OpenFlow13 dump-flows s1
ovs-ofctl -O OpenFlow13 dump-flows s2
```

The driver takes the first table dump before pingAll and the ten h1-h5 pings, then takes the final dump after those tests. It runs the same controller/topology and invokes these flow-table commands through Mininet.

Before tests: destination rules and broadcast rule already present

```
s1: n_packets=0, n_bytes=0, priority=10,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
s1: n_packets=0, n_bytes=0, priority=10,dl_dst=00:00:00:00:00:05 actions=output:"s1-eth4"
s1: n_packets=0, n_bytes=0, priority=5,dl_dst=ff:ff:ff:ff:ff:ff actions=FLOOD
s2: n_packets=0, n_bytes=0, priority=10,dl_dst=00:00:00:00:00:01 actions=output:"s2-eth3"
s2: n_packets=0, n_bytes=0, priority=10,dl_dst=00:00:00:00:00:05 actions=output:"s2-eth2"
s2: n_packets=0, n_bytes=0, priority=5,dl_dst=ff:ff:ff:ff:ff:ff actions=FLOOD
```

Selected Ryu output

```
Switch 1 connected
Installed: switch=1 dst=h5 out=4
All proactive rules installed on switch 1
Switch 2 connected
Installed: switch=2 dst=h5 out=2
All proactive rules installed on switch 2
```

After tests: h1/h5 forwarding rules and their packet counters

```
s1: n_packets=20, n_bytes=1736, priority=10,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
s1: n_packets=14, n_bytes=1372, priority=10,dl_dst=00:00:00:00:00:05 actions=output:"s1-eth4"
s2: n_packets=14, n_bytes=1260, priority=10,dl_dst=00:00:00:00:00:01 actions=output:"s2-eth3"
s2: n_packets=16, n_bytes=1568, priority=10,dl_dst=00:00:00:00:00:05 actions=output:"s2-eth2"
```

Connectivity and controller-workload results

```
PINGALL loss percent: 0.0
10 packets transmitted, 10 received, 0% packet loss, time 1804ms
rtt min/avg/max/mdev = 10.095/10.139/10.177/0.021 ms
Logged forwarding Packet-In events: 0
```

The h5 rules exist before testing with zero packet counters. Afterwards, those same matches and output ports have nonzero counters: s1 sends h5 traffic over port 4, and s2 sends it to host port 2. Rules for h1 carry replies in the other direction. This shows that the preinstalled rules actually forwarded packets.

Both switches also have a broadcast rule before the test. There is no CONTROLLER action in the proactive tables, so normal forwarding does not need Packet-In handling. The two successful ping checks provide the connectivity evidence alongside the installation log.

2. Link-metric-based path selection

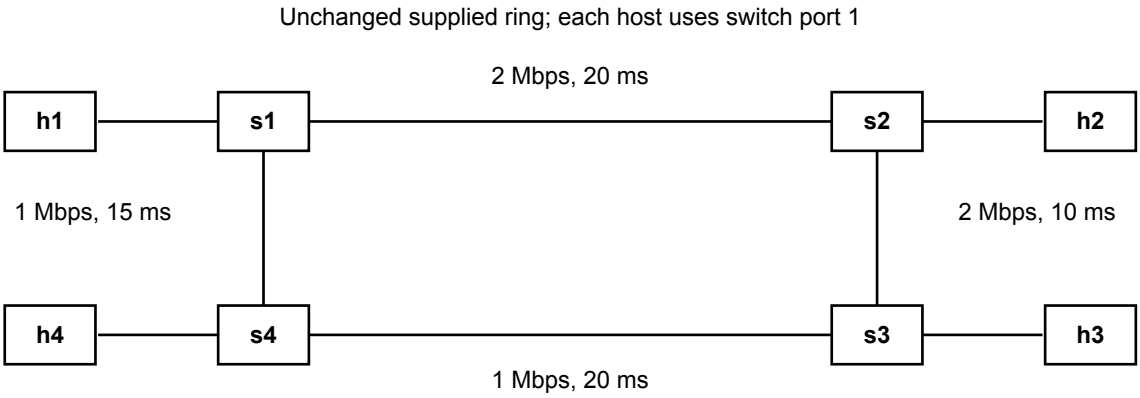


Figure 2. Parts 2 and 3 use this four-switch ring. Link labels give bandwidth and delay.

We chose delay as the static metric. The controller in p2/routing.py stores the ring in a NetworkX graph and uses the sum of link delays as path cost. The host MAC addresses and switch ports are fixed to the supplied topology: h1-h4 connect to port 1 of s1-s4. This keeps the implementation small; automatic topology or host discovery is not needed for this fixed experiment.

For each new source/destination MAC pair and EtherType, the controller finds the lowest-cost path and installs OpenFlow 1.3 rules along it. Reply traffic independently installs the reverse direction. Rules expire after five idle seconds. Broadcast ARP is sent directly to the other host ports by Ryu, rather than flooded around the ring, so there is no broadcast loop.

Test	Cost through s2	Cost through s4	Chosen h1-h3 route
Normal costs	20 + 10 = 30 ms	15 + 20 = 35 ms	s1 -> s2 -> s3
DELAY12=40	40 + 10 = 50 ms	15 + 20 = 35 ms	s1 -> s4 -> s3

In the second test, DELAY12=40 changes only the controller's s1-s2 cost. The physical topology stays unchanged. This is a link-cost-change test: the controller should prefer the other route because its total cost is now smaller.

Measured result	Normal costs	Changed cost
pingAll loss	0%	0%
Mean h1-h3 RTT (ms)	61.037	71.010

Selected controller log lines (utilization dictionaries omitted here):

```
PATH 1 -> 3: [1, 2, 3] adaptive=False
PATH 3 -> 1: [3, 2, 1] adaptive=False
PATH 1 -> 3: [1, 4, 3] adaptive=False
PATH 3 -> 1: [3, 4, 1] adaptive=False
```

The unloaded round-trip delays are close to twice the physical one-way path delay: about 60 ms through s2 and 70 ms through s4. The cost change therefore selects the expected path, and the measured RTT follows the physical route. Both forward and reverse paths are present in the logs; pingAll also checks every host pair in both directions.

3. Congestion-aware routing

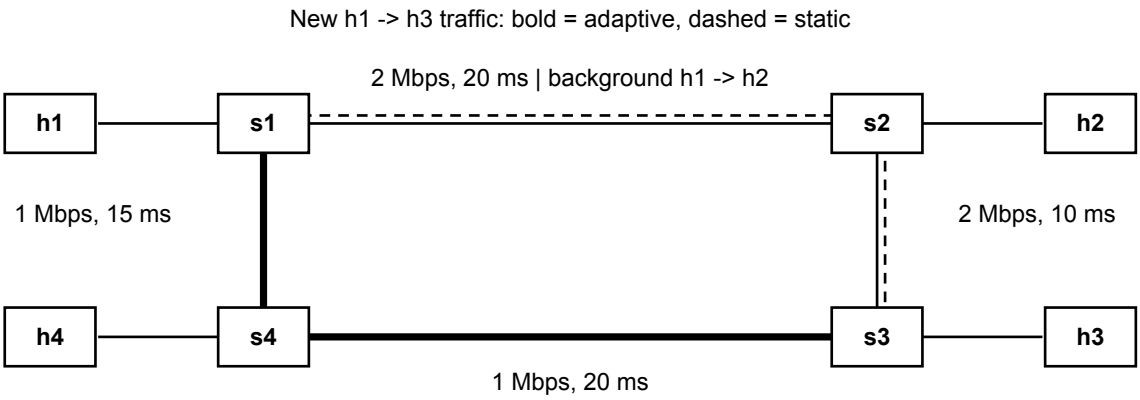


Figure 3. The background flow loads s1-s2. The later h1-h3 traffic can use the lower route.

Part 3 enables CONGESTION=1 in the same controller. Ryu requests port statistics once per second. For each inter-switch port we calculate utilization from the change in transmitted bytes:

Utilization rule

utilization = 8 x (new tx_bytes - old tx_bytes) / (elapsed seconds x link capacity in bits/s)

link utilization = max(utilization in the two directions)

if link utilization > 0.70: routing cost = link delay + 100 ms

otherwise: routing cost = link delay

We ran the same workload once with static routing and once with congestion-aware routing. After pingAll, we waited six seconds for the warm-up rules to expire. A h1-h2 UDP flow, configured for 24 seconds, used iperf -b 1.8M (about 1.89 Mbps) to load the 2 Mbps s1-s2 link. The script stops it after the foreground measurement. Five seconds later, we sent five h1-h3 pings and started an eight-second h1-h3 UDP flow at 600K bits/s. The nominal UDP rate is about 0.6 Mbps; iperf uses binary K units, so its offered rate is approximately 0.614 Mbps.

The h1-h3 host pair is new at this stage. It gets a path using the current measurements, while the active h1-h2 rules stay in place. Rules match a host pair and EtherType, so the UDP traffic reuses the path chosen for the preceding h1-h3 ping. A later flow for the same pair can choose again after five idle seconds.

Routing mode	h1-h3 route	Mean RTT (ms)	Receiver Mbps
Static	s1 -> s2 -> s3	61.698	0.480
Congestion-aware	s1 -> s4 -> s3	71.776	0.617

Receiver throughput is the mean of the first seven complete one-second server intervals in each run. We use the same window in both modes and exclude the final UDP summary/ACK, which can be delayed or lost under load. Both runs passed pingAll and the later ping test without loss.

At the adaptive h1-h3 decision, s1 port 2 utilization was 0.97, above the 0.70 threshold. The upper route therefore costs at least 130 ms, compared with 35 ms through s4. The alternate route delivered more of the foreground traffic in this run, although its propagation delay is longer.

One limitation is the one-second sampling interval: a short burst may finish before it is detected. OVS byte counters can include traffic waiting in the shaped egress queue, so the estimate may reflect offered load rather than exact wire utilization. There is no hysteresis, so paths can alternate near the threshold. We do not reroute active flows, and the fixed host-pair rules do not distinguish individual TCP or UDP sessions.

4. Selected flow tables and receiver logs

The Part 1 before/after tables and controller logs appear on pages 2-3. The evidence below covers Parts 2 and 3. Full unedited flow dumps and logs are in results/.

The rows below are parsed from the final flow tables. All rules match IPv4, source MAC 00:00:00:00:00:01 and destination MAC 00:00:00:00:00:03, with priority 10 and idle timeout 5 seconds. Packet counters confirm that the selected switches forwarded the traffic.

Run	Switch	Packets	Installed output action
p2-base	s1	8	output:"s1-eth2"
p2-base	s2	8	output:"s2-eth3"
p2-base	s3	8	output:"s3-eth1"
p2-changed	s1	8	output:"s1-eth3"
p2-changed	s3	8	output:"s3-eth1"
p2-changed	s4	8	output:"s4-eth2"
p3-static	s1	619	output:"s1-eth2"
p3-static	s2	411	output:"s2-eth3"
p3-static	s3	411	output:"s3-eth1"
p3-adaptive	s1	429	output:"s1-eth3"
p3-adaptive	s3	426	output:"s3-eth1"
p3-adaptive	s4	428	output:"s4-eth2"

Part 3 controller decisions (utilization dictionaries omitted):

```
PATH 1 -> 3: [1, 2, 3] adaptive=False
PATH 1 -> 3: [1, 4, 3] adaptive=True
```

p3-static receiver: first two intervals (iperf output)

```
[ 1] 0.0000-1.0000 sec 60.3 KBytes 494 Kbits/sec 5.029 ms 0/42 (0%)
[ 1] 1.0000-2.0000 sec 58.9 KBytes 482 Kbits/sec 5.424 ms 0/41 (0%)
```

p3-adaptive receiver: first two intervals (iperf output)

```
[ 1] 0.0000-1.0000 sec 77.5 KBytes 635 Kbits/sec 0.008 ms 0/54 (0%)
[ 1] 1.0000-2.0000 sec 74.6 KBytes 612 Kbits/sec 0.014 ms 0/52 (0%)
```

The static receiver can continue draining queued packets after the client stops sending; its final UDP acknowledgement may also time out. This is why the comparison uses the complete early receiver intervals rather than treating the offered client rate as received throughput.

5. Running and checking the submission

The experiments were rerun in an isolated copy with a new virtual environment. The tested system uses Linux, Python 3.12.3, Open vSwitch 3.3.9, Ryu 4.34, Mininet and iperf 2.1.9. Both supplied topology files and both Part 1 controllers are unchanged.

From the extracted code-a3 folder:

Setup and complete experiment run

```
sudo apt install mininet openvswitch-switch iperf python3-venv
python3 -m venv .venv
.venv/bin/pip install setuptools==67.6.1 wheel
.venv/bin/pip install -r requirements.txt
sudo systemctl start openvswitch-switch
sudo python3 experiments/run.py "$PWD/.venv/bin/ryu-manager"
python3 experiments/verify.py
```

The experiment script runs six cases and replaces results/. A failed connectivity check stops the run. Mininet runs with system Python and sudo; Ryu runs from the virtual environment. Stop other controllers before running. Use sudo mn -c after an interrupted Mininet run.

For an interactive run, start Ryu and the topology in separate terminals. All controllers listen on port 6633:

Part	Controller command after .venv/bin/ryu-manager	Topology command
1 reactive	--ofp-tcp-listen-port 6633 p1/reactive.py	sudo python3 p1_topo.py
1 proactive	--ofp-tcp-listen-port 6633 p1/proactive.py	sudo python3 p1_topo.py
2	--ofp-tcp-listen-port 6633 p2/routing.py	sudo python3 p3_topo.py
3	Prefix the Part 2 command with CONGESTION=1	sudo python3 p3_topo.py

For the changed-cost Part 2 case, prefix the controller command with DELAY12=40. Inside Mininet, use pingall and h1 ping -c 10 h3. Inspect a switch with sh ovs-ofctl -O OpenFlow13 dump-flows s1.

Requirement checked	Evidence in this submission
Part 1: both approaches, connectivity, comparison and trade-off	Controllers in p1/; results/p1-*; pages 1-3
Part 2: graph, host ports, delay costs, bidirectional routes and changed-cost test	p2/routing.py; results/p2-*; pages 4 and 6
Part 3: statistics, threshold rule, later-flow routing and static comparison	CONGESTION=1; results/p3-*; pages 5 and 6
Source, topologies, setup, report and correctly named archive	All required files are included under code-a3/

The bundled Ryu wheel is third-party Apache-2.0 software, with the existing eventlet ALREADY_HANDLED compatibility fix used on this machine. It removes the dependency on an external local Ryu source directory. pip downloads its other dependencies. SETUP.txt includes individual commands. Report regeneration additionally needs ReportLab and installed Arial fonts; neither is needed to run the networking experiments.